LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

Disusun oleh :

Nama : syafa sartika putri aryanti

NPM : 25782056

Kelas : TRI-3B

Dosen:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

5.6 Latihan

- **Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):** Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (sendStatus == 0), lakukan eksperimen pemutusan simpul:
  i. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
  ii. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?
  iii. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

  Jawab :



Hasil Observasi: Ya, status pengiriman untuk MAC address board yang dicabut dayanya (2c:f4:32:6a:7d:34) terbukti berganti menjadi **Status: Gagal (Tidak Terjangkau)**, sementara board penerima lainnya yang masih menyala tetap berstatus Berhasil Diterima.

Penjelasan Teknis (Layer 2 MAC & ACK): Meskipun ESP-NOW tidak menggunakan protokol berbasis sambungan seperti TCP (di *Transport Layer*), ESPNOW bekerja langsung pada **Layer 2 (Data Link / MAC Layer)** dari model OSI. Pada lapisan perangkat keras Wi-Fi standar, setiap kali paket data jenis *unicast* dikirimkan, chip pemancar secara otomatis mengharapkan sinyal konfirmasi fisik berupa **ACK (*Acknowledgment*)** dari perangkat tujuan. Ketika *board* penerima dicabut dari sumber daya, ia mati dan tidak dapat mengirimkan sinyal balasan ACK. Setelah chip pengirim gagal menerima ACK hingga batas maksimal percobaan ulang (*retry*), sistem mendeteksi bahwa perangkat tujuan tidak merespons, sehingga memicu fungsi *callback* (OnDataSent) untuk mencatat status gagal atau tidak terjangkau.

- **Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):** Jalankan jaringan painlessMesh pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID: ....

i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback* changedConnectionCallback() untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?

ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

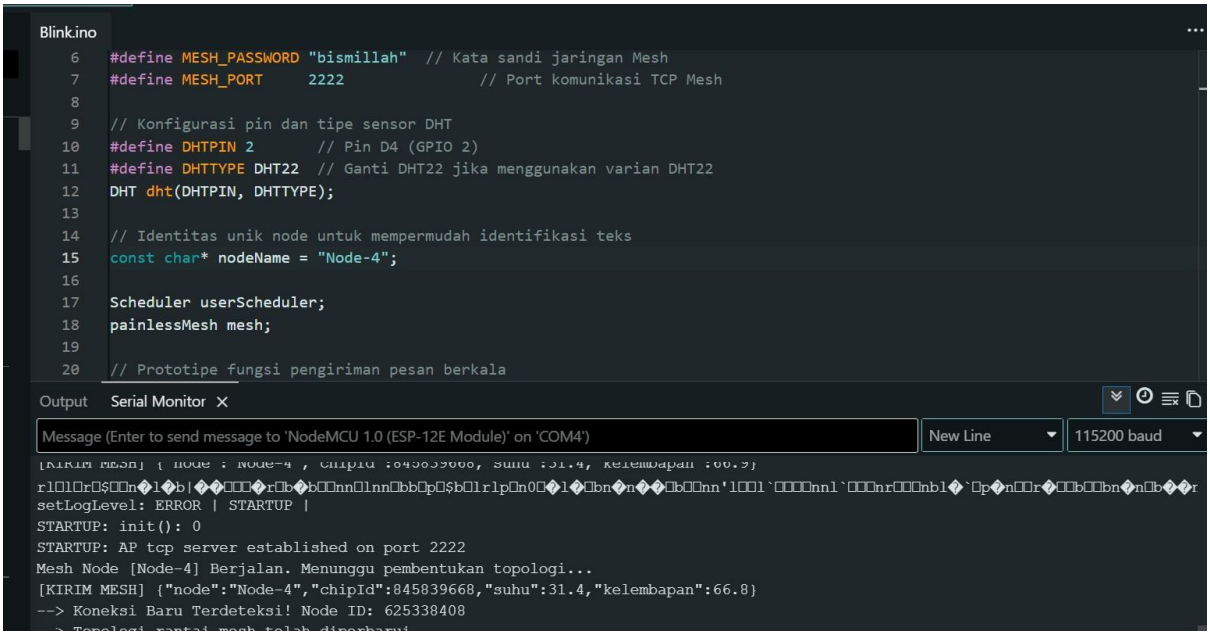
Jawaban :

Deteksi Simpul Terputus (changedConnectionCallback):

- Ketika salah satu board dimatikan atau ditekan tombol *RST*, *Serial Monitor* pada node yang masih menyala akan mendeteksi hilangnya koneksi secara otonom.
- Berdasarkan observasi waktu, fungsi *callback* biasanya membutuhkan waktu sekitar **3 hingga 10 detik** untuk mengenali bahwa suatu node telah keluar dari rantai mesh (tergantung pada siklus *heartbeat* jaringan).

Waktu Pemulihan Otomatis (*Self-Healing* / Bergabung Kembali):

- Saat daya pada board yang dicabut disambungkan kembali, node tersebut langsung melakukan *scanning* dan memulihkan koneksinya.
- Waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh (ditandai dengan munculnya kembali pesan --> Koneksi Baru Terdeteksi!) berkisar antara **2 hingga 5 detik** tanpa perlu mengubah atau mengonfigurasi ulang jaringan, karena parameter MESH_PREFIX dan MESH_PASSWORD sudah tertanam di dalam programnya.

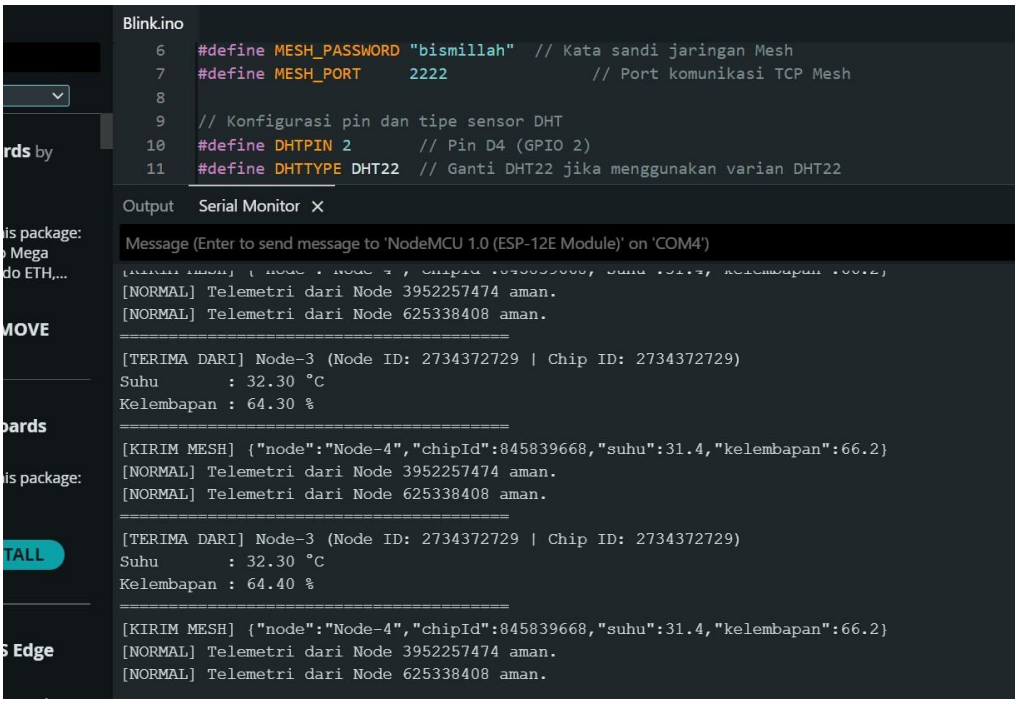


- **Latihan 3 (Eksperimen Modifikasi Filter Telemetri & Ambang Batas JSON - Praktikum 4):** Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetri dari semua pengirim tanpa penyaringan.

i. Modifikasilah fungsi receivedCallback pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim). ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetri dari Node <chipId> aman.

iii. Tuliskan blok fungsi receivedCallback(uint32_t from, String &msg) hasil modifikasi Anda!

Jawaban :



Pengujian Latihan 3 dilakukan untuk mengimplementasikan logika penyaringan data telemetri (*telemetry data filtering*) serta penentuan ambang batas suhu (*threshold logic*) pada fungsi panggilan balik receivedCallback(). Modifikasi kode ini bertujuan agar simpul (*node*) penerima dapat membedakan penanganan respons antara data kondisi suhu normal dan kondisi suhu kritis secara otomatis tanpa membanjiri antarmuka *Serial Monitor*.

Berdasarkan hasil pengujian yang terekam pada *Serial Monitor*, sistem berhasil memproses dan memfilter data telemetri yang diterima dari beberapa simpul tetangga secara *real-time*:

- **Kondisi Suhu Normal (≤ 32.0°C):** Ketika simpul tetangga (seperti Node 3952257474 dan Node 625338408) melaporkan suhu di bawah atau sama dengan 32.0°C, program mencetak respons ringkas satu baris berformat [NORMAL] Telemetri dari Node <chipId> aman.
- **Kondisi Suhu Kritis (> 32.0°C):** Ketika simpul tetangga melaporkan suhu melebihi ambang batas 32.0°C, sistem secara otomatis mencetak kartu telemetri penuh. Hal ini

terbukti pada data yang diterima dari Node-3 (Node ID / Chip ID terkait) yang mendeteksi suhu sebesar 32.30°C, di mana seluruh rincian nama simpul, ID, nilai suhu, serta kelembapan ditampilkan secara utuh dalam blok terpisah.
Pengujian ini membuktikan bahwa penambahan logika penapis (*conditional filter*) pada fungsi receivedCallback() efektif dalam mengoptimalkan efisiensi pemrosesan data pada *edge node*. Sistem mampu memprioritaskan visualisasi data darurat (*warning alerts*) sekaligus mereduksi kerapatan log (*log flooding*) saat kondisi jaringan berada pada status aman.